

Updated Python Practical Programs (All 14 Practicals)

1. Matrix Operations using NumPy

```
import numpy as np

A = np.array([[1, 2],
              [3, 4]])

B = np.array([[5, 6],
              [7, 8]])

print("Addition:")
print(A + B)

print("Multiplication:")
print(np.dot(A, B))

print("Determinant of A:")
print(np.linalg.det(A))

print("Inverse of A:")
print(np.linalg.inv(A))

eigenvalues, eigenvectors = np.linalg.eig(A)

print("Eigenvalues:")
print(eigenvalues)

print("Eigenvectors:")
print(eigenvectors)
```

2. PCA

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA

X = np.array([
    [2, 3],
    [3, 4],
    [4, 5],
    [5, 6]
])

plt.scatter(X[:,0], X[:,1])
plt.title("Original Data")
plt.show()

pca = PCA(n_components=1)

reduced = pca.fit_transform(X)

print("Reduced Data:")
print(reduced)

print("Explained Variance Ratio:")
print(pca.explained_variance_ratio_)
```

3. Naive Bayes Spam Classifier

```
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
messages = [
```

```

        "Win money now",
        "Claim your prize",
        "Hello friend",
        "Meeting tomorrow"
    ]

    labels = [
        "spam",
        "spam",
        "ham",
        "ham"
    ]

    vectorizer = CountVectorizer()

    X = vectorizer.fit_transform(messages)

    model = MultinomialNB()

    model.fit(X, labels)

    test = vectorizer.transform(["Win prize now"])

    prediction = model.predict(test)

    print("Prediction:", prediction[0])

```

4. Coin Toss & Poisson Simulation

```

import numpy as np
import matplotlib.pyplot as plt

tosses = np.random.choice(['H', 'T'], size=100)

heads = np.sum(tosses == 'H')
tails = np.sum(tosses == 'T')

print("Heads:", heads)
print("Tails:", tails)

plt.bar(['Heads', 'Tails'], [heads, tails])

plt.title("Coin Toss Simulation")

plt.show()

data = np.random.poisson(lam=5, size=1000)

print("Mean:", np.mean(data))

plt.hist(data)

plt.title("Poisson Distribution")

plt.show()

```

5. Dice Roll & Binomial Simulation

```

import numpy as np
import matplotlib.pyplot as plt

rolls = np.random.randint(1, 7, size=100)

counts = []

for i in range(1,7):
    count = np.sum(rolls == i)
    counts.append(count)

    print("Count of", i, "=", count)

plt.bar([1,2,3,4,5,6], counts)

```

```

plt.title("Dice Roll Simulation")
plt.show()

data = np.random.binomial(n=10, p=0.5, size=1000)
print("Mean:", np.mean(data))
plt.hist(data)
plt.title("Binomial Distribution")
plt.show()

```

6. Sampling Techniques

```

import pandas as pd

data = pd.DataFrame({
    'Patient_ID': [1, 2, 3, 4, 5, 6, 7, 8],
    'Waiting_Time': [10, 15, 20, 25, 30, 35, 40, 45]
})

print("Original Data:")
print(data)

sample = data.sample(n=4, random_state=1)

print("Random Sample:")
print(sample)

```

7. Z-Test for Mean

```

import numpy as np

sample_mean = 52
population_mean = 50
sigma = 5
n = 100

z = (sample_mean - population_mean) / (sigma / np.sqrt(n))

print("Z value =", z)

if abs(z) > 1.96:
    print("Reject Null Hypothesis")
else:
    print("Fail to Reject Null Hypothesis")

```

8. t-Test

```

import numpy as np

sample_mean = 22
population_mean = 20
s = 4
n = 10

t = (sample_mean - population_mean) / (s / np.sqrt(n))

print("t value =", t)

if abs(t) > 2.262:
    print("Reject Null Hypothesis")
else:
    print("Fail to Reject Null Hypothesis")

```

9. Z-Test for Proportion

```
import numpy as np

p_hat = 0.58
p = 0.50
n = 200

z = (p_hat - p) / np.sqrt((p * (1-p)) / n)

print("Z value =", z)

if abs(z) > 1.96:
    print("Reject Null Hypothesis")
else:
    print("Fail to Reject Null Hypothesis")
```

10. Chi-Square Test

```
import numpy as np
from scipy.stats import chi2_contingency

data = np.array([
    [30, 20],
    [25, 35]
])

chi2, p, dof, expected = chi2_contingency(data)

print("Chi-Square value =", chi2)
print("p value =", p)

if p < 0.05:
    print("Variables are dependent")
else:
    print("Variables are independent")
```

11. Linear Regression (Food Delivery Revenue)

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

orders = np.array([10, 20, 30, 40, 50]).reshape(-1,1)
revenue = np.array([1000, 2000, 3000, 4000, 5000])

model = LinearRegression()
model.fit(orders, revenue)

predicted = model.predict(orders)

print("Slope:", model.coef_[0])
print("Intercept:", model.intercept_)

plt.scatter(orders, revenue)
plt.plot(orders, predicted)

plt.title("Orders vs Revenue")
plt.show()
```

12. Linear Regression (Rainfall vs Water Level)

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

rainfall = np.array([10,20,30,40,50,60,70,80,90,100]).reshape(-1,1)
water_level = np.array([2,4,5,7,8,10,11,13,14,16])

model = LinearRegression()
model.fit(rainfall, water_level)
predicted = model.predict(rainfall)
plt.scatter(rainfall, water_level)
plt.plot(rainfall, predicted)
plt.title("Rainfall vs Water Level")
plt.show()
```

13. Directional Derivative

```
import numpy as np

gradient = np.array([2, 3])
direction = np.array([1, 1])
unit_direction = direction / np.linalg.norm(direction)
directional_derivative = np.dot(gradient, unit_direction)
print("Directional Derivative =", directional_derivative)
```

14. Gradient Descent Algorithm

```
import numpy as np
import matplotlib.pyplot as plt

x = 10
learning_rate = 0.1
values = []

for i in range(50):
    gradient = 2 * x
    x = x - learning_rate * gradient
    values.append(x)

print("Minimum value near:", x)
plt.plot(values)
plt.title("Gradient Descent Convergence")
plt.show()
```